

● CYBERSECURITY EBOOK

WIRESHARK

The Complete Guide

Master Packet Analysis, Network Forensics &
Threat Detection — Step by Step

v4.x Edition

2024 – 2025

Written by

Tanish Sareen

Cybersecurity & Forensics Specialist | BCA – Cybersecurity

Published June 2026

TABLE OF CONTENTS

1	Introduction to Wireshark
2	Installation & Setup
3	The Wireshark Interface — Deep Dive
4	Capturing Packets
5	Capture Filters — Berkley Packet Filter (BPF)
6	Display Filters — Master Reference
7	Protocol Analysis — TCP/IP Stack
8	Following Streams
9	HTTP & HTTPS Analysis
10	DNS Analysis
11	ARP, ICMP & Network Layer Analysis
12	Wireless (Wi-Fi) Packet Analysis
13	Statistics & I/O Graphs
14	Wireshark for Security — Attack Detection
15	Malware Traffic Analysis
16	TShark — Command Line Wireshark
17	Exporting, Reporting & Evidence Handling
18	Advanced Features & Plugins
19	Real-World Labs & CTF Scenarios
20	Cheat Sheet & Quick Reference

Introduction to Wireshark

Wireshark is the world's most widely used open-source network protocol analyser. Originally created by Gerald Combs in 1998 under the name **Ethereal**, it was renamed Wireshark in 2006. Today it is the de-facto standard for packet-level network inspection, trusted by security professionals, forensic analysts, network engineers, and students worldwide.

1.1 What is a Packet Sniffer?

Every piece of data that travels across a network is broken into small chunks called **packets**. Each packet carries a header (metadata: source IP, destination IP, protocol, etc.) and a payload (the actual data). A packet sniffer — or protocol analyser — intercepts these packets in real time and decodes them so humans can read them.

1.2 Why Wireshark?

- **Free & Open Source** — licensed under GNU GPL v2.
- **Cross-platform** — Windows, macOS, Linux, FreeBSD.
- **Deep Protocol Support** — decodes over 3,000 network protocols natively.
- **Live Capture** — captures traffic in real time from any network interface.
- **Offline Analysis** — open and analyse pre-recorded .pcap / .pcapng files.
- **Powerful Filtering** — dual filter system (capture filters + display filters).
- **Export & Reporting** — export to CSV, JSON, XML, Plain Text, or pcap.
- **TShark CLI** — headless command-line version for scripting and automation.
- **Massive Community** — documentation, plugins, and active development.

1.3 Core Use Cases

Network Troubleshooting

Diagnose slow applications, dropped connections, duplicate ACKs, retransmissions, DNS resolution failures, and misconfigurations at the packet level.

Security Analysis & Forensics

Detect port scans, ARP poisoning, MITM attacks, lateral movement, data exfiltration, malware C2 beacons, and intrusion attempts by inspecting raw traffic.

Protocol Development

Developers use Wireshark to validate that custom protocols behave correctly and that implementations conform to RFC standards.

CTF & Ethical Hacking

Capture the Flag competitions routinely include .pcap challenges. Wireshark is essential for extracting credentials, reconstructing files, and discovering flags hidden in traffic.

Learning & Education

Nothing teaches networking like watching packets fly. Wireshark makes abstract TCP/IP concepts concrete and visible.

■ NOTE

Sniffing traffic on networks you do not own or have written permission to monitor is illegal in most jurisdictions. Always operate in a lab environment, your own network, or with explicit written authorisation. Wireshark displays a reminder each time it captures on a promiscuous interface.

Installation & Setup

2.1 Installing on Windows

Wireshark for Windows ships as a self-contained installer that also bundles **Npcap** (the packet-capture driver) and the optional **USBPcap** component.

- Visit <https://www.wireshark.org/download.html>
- Download the **Windows x64 Installer** (.exe)
- Run the installer — accept the licence agreement.
- On the *Choose Components* screen leave all defaults ticked.
- On the *Packet Capture* screen ensure "**Install Npcap**" is checked.
- On the *USB Capture* screen tick USBPcap only if you need USB analysis.
- Complete the wizard — reboot may be required for Npcap.
- Launch Wireshark from the Start Menu.

■ PRO TIP

Npcap replaces the old WinPcap. Always use the bundled Npcap — do not install

WinPcap separately, as it is unmaintained and incompatible with modern Windows.

2.2 Installing on macOS

- Download the **.dmg** installer from [wireshark.org](https://www.wireshark.org).
- Open the **.dmg** and drag Wireshark to Applications.
- On first launch macOS will prompt to install the ChmodBPF package — click *Install*.
- Alternatively, use Homebrew: **brew install --cask wireshark**
- Grant the app permission to capture in System Settings → Privacy & Security if needed.

2.3 Installing on Linux (Debian / Ubuntu)

```
# Update package lists  
  
sudo apt update
```

```
# Install Wireshark

sudo apt install wireshark -y

# When prompted: 'Should non-superusers be able to capture packets?' → YES

# Add your user to the wireshark group

sudo usermod -aG wireshark $USER

# Log out and back in for group membership to take effect

newgrp wireshark

# Launch

wireshark &
```

2.4 Installing on Kali Linux

Wireshark is pre-installed on Kali Linux. To update to the latest version:

```
sudo apt update && sudo apt install wireshark -y

# Verify version

wireshark --version
```

2.5 Installing TShark Only (Headless Servers)

```
# Debian/Ubuntu

sudo apt install tshark -y

# CentOS/RHEL/Fedora

sudo dnf install wireshark-cli -y

# Verify

tshark --version
```

2.6 Verifying the Installation

After launching Wireshark you should see the welcome screen listing all available network interfaces with a small sparkline graph showing live traffic. If interfaces are missing, the capture driver (Npcap / libpcap) is not installed correctly.

■ WARNING

On Linux, if you run Wireshark as root (`sudo wireshark`) it works but is a security risk.

The correct approach is adding your user to the 'wireshark' group as shown above.

The Wireshark Interface — Deep Dive

Understanding every panel and menu in Wireshark is fundamental. Let's walk through the entire GUI methodically.

3.1 Welcome / Start Screen

The welcome screen shows:

- **Interface List** — every network adapter on the system (Ethernet, Wi-Fi, Loopback, Bluetooth).
- **Traffic Sparklines** — live mini-graphs showing activity on each interface, helping you pick the right one.
- **Capture Filter Bar** — type a BPF filter before starting a capture.
- **Recent Files** — quick access to recently opened .pcap files.
- **Open / Capture Menu** — access to Open, Close, Merge pcap operations.

3.2 The Main Capture Window — Five Core Panels

Panel 1: The Toolbar

The toolbar provides one-click access to the most common actions:

Icon / Button	Function
■ Fin (blue shark)	Start a new capture (shows interface chooser)
■ Red square	Stop current capture
■ Restart	Stop and immediately restart capture
■ Open	Open a saved .pcap / .pcapng file
■ Save	Save current capture to file
■ Find	Open Find Packet dialog (Ctrl+F)
← / →	Navigate previous / next packet in selection history
■ / ■ First/Last	Jump to first or last packet
■ Zoom In/Out	Resize font in packet list

■ Coloring	Toggle packet coloring rules on/off
■ Autoscroll	Auto-scroll to newest packet during live capture

Panel 2: The Display Filter Bar

Directly below the toolbar. This is where you type **display filters** to show only packets matching your criteria. The bar turns **green** for a valid filter, **red** for invalid syntax, and **yellow** for a partial/warning state. Pressing **Enter** applies the filter. Click **X** to clear. Click the bookmark icon to save frequently used filters.

Panel 3: Packet List Panel (Top)

A scrollable table of every packet captured. Default columns:

- **No.** — packet number (sequential from capture start).
- **Time** — timestamp (configurable: relative, absolute, UTC, delta).
- **Source** — source IP address (or MAC for Layer 2).
- **Destination** — destination IP address.
- **Protocol** — highest-layer protocol Wireshark has decoded.
- **Length** — total packet length in bytes.
- **Info** — human-readable summary of the packet's key information.

Each row is colour-coded by protocol (configurable via View → Coloring Rules).

Panel 4: Packet Details Panel (Middle)

Click any packet in the list to expand its protocol layers here. Each layer is a collapsible tree node:

- **Frame** — physical capture metadata (interface, encapsulation, timestamps).
- **Ethernet II** — Layer 2 frame with source/destination MAC addresses and EtherType.
- **Internet Protocol v4/v6** — Layer 3 header: TTL, flags, IDs, source/dest IPs.
- **TCP / UDP / ICMP** — Layer 4 transport header with port numbers, flags, checksums.
- **Application Protocol** — HTTP, DNS, TLS, SMTP, FTP decoded in human-readable form.

Right-clicking any field gives options: Apply as Filter, Prepare Filter, Copy Value, Follow Stream, Wiki Protocol Page, and more.

Panel 5: Packet Bytes Panel (Bottom)

The raw hexadecimal dump of the selected packet. Left side shows hex bytes; right side shows ASCII interpretation. When you click a field in the Details panel, the corresponding bytes highlight in blue here — extremely useful for understanding exactly where each header field sits in the raw packet.

3.3 The Menu Bar

File Menu

- **Open** — load a .pcap, .pcapng, .cap, .log or other supported file.
- **Open Recent** — last 10 files.
- **Merge** — combine two or more pcap files into one.
- **Import from Hex Dump** — reconstruct packets from hex text.
- **Close** — close current file (keeps Wireshark open).
- **Save / Save As** — save in pcapng (default), pcap (legacy), CSV, JSON, XML, PSML, PDML.
- **Export Specified Packets** — save a subset by display filter.
- **Export Objects** — extract HTTP, SMB, DICOM, TFTP objects from the stream.
- **Print** — print packet details.
- **Quit** — exit Wireshark.

Edit Menu

- **Find Packet (Ctrl+F)** — search by display filter, hex string, or regex.
- **Mark / Unmark Packet** — highlight packets for reference (black background).
- **Ignore Packet** — exclude from analysis without deleting.
- **Set Time Reference** — pin a packet as time zero (useful for latency analysis).
- **Copy** — copy selected field value, row summary, or all fields.
- **Preferences (Ctrl+Shift+P)** — global settings for capture, UI, protocols.

View Menu

- **Coloring Rules** — manage the colour-coding scheme.
- **Reload** — re-read the capture file from disk.
- **Packet List / Details / Bytes** — toggle visibility of each panel.
- **Name Resolution** — enable/disable DNS, MAC OUI, port name resolution.
- **Internals** — dissector tables, supported protocols list.

Capture Menu

- **Options (Ctrl+K)** — full capture options dialog.
- **Start / Stop / Restart** — control the current capture.
- **Capture Filters** — manage saved BPF filter library.
- **Refresh Interfaces** — update interface list without restarting.

Analyse Menu

- **Display Filters** — manage saved display filter library.
- **Apply / Prepare as Filter** — build filter from selected field.
- **Follow Stream** — reconstruct TCP/UDP/TLS streams.

- **Expert Information** — summary of errors, warnings, notes, and chats.
- **Decode As** — force Wireshark to decode a port as a different protocol.
- **Enabled Protocols** — toggle individual protocol dissectors.

Statistics Menu

- **Summary** — capture file metadata, byte and packet counts.
- **Protocol Hierarchy** — tree view of all protocols and their byte share.
- **Conversations** — list of unique communication pairs (Ethernet, IP, TCP, UDP).
- **Endpoints** — per-address TX/RX statistics.
- **I/O Graphs** — time-series graphs of traffic volume.
- **TCP Stream Graph** — round-trip time, throughput, time-sequence graphs.
- **HTTP / DNS / VoIP Statistics** — protocol-specific dashboards.
- **Flow Graph** — visual ladder diagram of packet exchange.

Wireless Menu (Wi-Fi only)

- **WLAN Traffic** — 802.11 traffic statistics per BSSID.
- **Bluetooth ATT Server Attributes** — BLE attribute table.

Tools Menu

- **Firewall ACL Rules** — generate firewall rules from packet selection.
- **Lua Console** — interactive Lua scripting shell.
- **Credentials** — extract plaintext credentials from captures.

Help Menu

- **Contents (F1)** — offline user guide.
- **Wireshark Manual** — man page viewer.
- **Sample Captures** — curated .pcap examples on the Wireshark wiki.
- **About** — version, plugins, libraries, author credits.

3.4 Status Bar

The status bar at the bottom shows: expert-information indicator (circle colour = severity), applied display filter, selected packet field description, capture file path, packet count (displayed vs total), and profile name.

■ PRO TIP

Right-click any column header in the Packet List to add, remove, or reorder columns.

Useful additions: Absolute Time, Delta Time, TCP Stream Index, IP DSCP.

3.5 Profiles

Wireshark supports multiple configuration profiles (Edit → Configuration Profiles). Create separate profiles for: **Default**, **Security Analysis**, **VoIP**, **HTTP Debug**, etc. Each profile stores its own coloring rules, columns, display filters, and preferences independently.

Capturing Packets

4.1 Choosing the Right Interface

On the welcome screen, observe the sparkline graphs. Select the interface that shows live traffic. Common interfaces and their meanings:

Interface Name	Traffic Type
eth0 / Ethernet	Wired LAN traffic
wlan0 / Wi-Fi	Wireless 802.11 traffic (data frames in managed mode)
lo / Loopback	localhost traffic (127.0.0.1 — great for local app debugging)
any (Linux)	All interfaces simultaneously
en0 (macOS)	Primary network interface
USBPcap	USB device communication (requires USBPcap on Windows)
Pipe / FIFO	Feed from an external tool via named pipe

4.2 Starting a Basic Capture (Step by Step)

- **Step 1** — Launch Wireshark.
- **Step 2** — Double-click the desired interface in the welcome screen (or single-click then press the blue shark-fin button).
- **Step 3** — Watch packets scroll in the Packet List panel in real time.
- **Step 4** — Generate traffic: open a browser, ping a host, run a port scan.
- **Step 5** — Press the red square button (or Ctrl+E) to stop the capture.
- **Step 6** — Analyse the captured packets.
- **Step 7** — File → Save As to save as .pcapng (preferred) or .pcap (legacy).

4.3 Capture Options Dialog (Ctrl+K)

The Capture Options dialog gives fine-grained control. Key settings:

- **Interface** — select one or multiple interfaces to capture simultaneously.

- **Promiscuous Mode** — capture all packets on the segment, not just those addressed to your MAC. Enable by default for security analysis.
- **Monitor Mode (Wi-Fi)** — see raw 802.11 frames including management and control frames. Requires compatible adapter + driver.
- **Capture Filter (BPF)** — pre-filter at the kernel level to reduce load and file size.
- **Capture File** — specify a file path to write directly to disk (ring buffer capture).
- **Output Options** — enable ring buffer: create new file every N megabytes or N seconds.
- **Stop Capture Automatically** — after N packets, N megabytes, or N seconds.
- **Snaptlen** — bytes to capture per packet (default: 262144; set to 65535 for max compatibility).
- **Buffer Size** — kernel capture buffer in MB (increase for high-speed captures to avoid drops).
- **Resolve MAC / IP / Port Names** — toggle name resolution during capture.

4.4 Ring Buffer Capture (Long-Term Monitoring)

For continuous monitoring, use the ring buffer feature. Wireshark writes a series of files and wraps around, deleting the oldest when the count is reached:

```
# Example: Capture using TShark with a ring buffer

# Creates 10 files of 100 MB each, cycling continuously

tshark -i eth0 -b filesize:102400 -b files:10 -w /captures/ring.pcap
```

4.5 Remote Capture

Capture traffic on a remote Linux machine and pipe it to local Wireshark in real time:

```
# On Linux – pipe tcpdump output over SSH to local Wireshark

ssh user@remote-host 'tcpdump -i eth0 -U -w - not port 22' | wireshark -k -i -

# Explanation:

# -U → unbuffered output

# -w - → write to stdout

# not port 22 → exclude SSH traffic (prevents feedback loop)

# wireshark -k → start capturing immediately

# -i - → read from stdin
```

4.6 Capture in Promiscuous vs Monitor Mode

Promiscuous Mode: Your NIC receives all Ethernet frames on the collision domain (or VLAN), not just frames destined for your MAC address. Essential for analysing traffic to/from other hosts on a hub or port-mirrored switch.

Monitor Mode (RFMON): The wireless adapter captures all 802.11 frames in the air — including management (beacons, probe requests) and control frames — without associating to an AP. Required for Wi-Fi security analysis. On Linux, enable with:

```
sudo ip link set wlan0 down  
  
sudo iw dev wlan0 set type monitor  
  
sudo ip link set wlan0 up  
  
wireshark # now select wlan0 in monitor mode
```

Capture Filters — Berkeley Packet Filter (BPF)

Capture filters use Berkeley Packet Filter (BPF) syntax and run in the kernel before packets reach Wireshark. They reduce capture file size and CPU load. Unlike display filters, BPF syntax is more limited but extremely fast.

■ NOTE

Capture filters are applied BEFORE packets are recorded. Packets that do not match are discarded permanently. Display filters only hide/show packets — they never delete them.

5.1 Basic BPF Primitives

```
host 192.168.1.100 # Traffic to OR from this IP

src host 192.168.1.100 # Traffic FROM this IP only

dst host 192.168.1.100 # Traffic TO this IP only

net 192.168.1.0/24 # Entire /24 subnet

src net 10.0.0.0/8 # Traffic from 10.x.x.x

port 80 # TCP or UDP port 80

src port 443 # Source port 443

dst port 53 # Destination port 53 (DNS)

portrange 1-1024 # Port range

tcp # TCP traffic only

udp # UDP traffic only

icmp # ICMP traffic only

arp # ARP packets only

ether host AA:BB:CC:DD:EE:FF # MAC address filter
```



```
broadcast # Broadcast frames

multicast # Multicast frames

less 128 # Packets smaller than 128 bytes

greater 1000 # Packets larger than 1000 bytes

ip6 # IPv6 traffic only
```

5.2 Boolean Operators in BPF

```
# AND – both conditions must be true

host 192.168.1.100 and port 80

# OR – either condition must be true

port 80 or port 443

# NOT – negate a condition

not port 22

# Grouping with parentheses

host 10.0.0.5 and (port 80 or port 443)

# Complex example: All HTTP/HTTPS traffic except from one host

(port 80 or port 443) and not host 192.168.1.1
```

5.3 Essential Capture Filter Recipes

Goal	BPF Filter
Capture DNS only	<code>udp port 53</code>
Web traffic only	<code>port 80 or port 443</code>
Exclude SSH	<code>not port 22</code>
One host only	<code>host 192.168.1.50</code>
LAN subnet only	<code>net 192.168.0.0/16</code>

ICMP (ping) only	<code>icmp</code>
ARP storms	<code>arp</code>
Large packets (MTU issues)	<code>greater 1400</code>
SMB traffic	<code>port 445</code>
DHCP	<code>port 67 or port 68</code>
Between two hosts	<code>host 10.0.0.1 and host 10.0.0.2</code>
TCP SYN packets (port scans)	<code>tcp[tcpflags] == tcp-syn</code>
TCP RST packets	<code>tcp[tcpflags] & tcp-rst != 0</code>
Malformed / fragmented IP	<code>ip[6:2] & 0x1fff != 0</code>
VoIP (SIP + RTP)	<code>port 5060 or portrange 16384-32767</code>

5.4 Saving Capture Filters

- Go to **Capture** → **Capture Filters**.
- Click the **+** button to add a new entry.
- Name the filter (e.g., 'Web Only') and paste the BPF expression.
- Click **OK** — it is now available in the filter dropdown anytime.

Display Filters — Master Reference

Display filters use Wireshark's own rich filter language — entirely separate from BPF. They can reference any decoded protocol field by name, support comparison operators, boolean logic, regular expressions, and membership sets. They never drop packets — only hide them temporarily.

6.1 Filter Syntax Rules

- Protocol names are lowercase: **tcp**, **http**, **dns**, **tls**.
- Fields are dot-separated: **ip.src**, **tcp.port**, **http.request.method**.
- Comparison operators: **==**, **!=**, **<**, **>**, **<=**, **>=**.
- Boolean: **and** (or **&&**), **or** (or **||**), **not** (or **!**).
- Strings need quotes: **http.host == "example.com"**.
- Existence test (field present): just the field name, e.g., **http.cookie**.
- Regex: **matches** operator, e.g., **http.host matches "evil.*\ru"**.
- Membership: **in** operator, e.g., **tcp.port in {80 443 8080}**.
- Slice: **tcp[0:2]** — first 2 bytes of TCP header.
- Bitmasking: **tcp.flags & 0x02** — test individual bits.

6.2 Essential Display Filters by Category

IP Address Filters

```
ip.addr == 192.168.1.100 # To or from this IP

ip.src == 10.0.0.1 # Only from this IP

ip.dst == 8.8.8.8 # Only to this IP

ip.addr == 192.168.1.0/24 # Entire subnet

ip.src == 192.168.1.0/24 and ip.dst == 8.8.8.8

!(ip.addr == 192.168.1.1) # Exclude this IP

ip.ttl < 10 # Low TTL (may indicate traceroute or spoofing)

ip.flags.df == 1 # Don't-Fragment bit set
```

```
ip.flags.mf == 1 # More Fragments (fragmented packets)

ip.frag_offset > 0 # Non-first fragments
```

TCP Filters

```
tcp.port == 443 # Either direction port 443

tcp.srcport == 80 # Source port 80

tcp.dstport == 22 # Destination port 22 (SSH)

tcp.flags.syn == 1 # SYN packets

tcp.flags.syn == 1 and tcp.flags.ack == 0 # SYN only (new connections)

tcp.flags.rst == 1 # RST (connection reset)

tcp.flags.fin == 1 # FIN (connection close)

tcp.flags == 0x002 # Pure SYN (hex flag mask)

tcp.flags == 0x012 # SYN-ACK

tcp.analysis.retransmission # Retransmitted packets

tcp.analysis.duplicate_ack # Duplicate ACKs

tcp.analysis.lost_segment # Lost segments

tcp.analysis.zero_window # TCP Zero Window (flow control stall)

tcp.window_size == 0 # Window size zero

tcp.len > 0 # TCP segments with payload

tcp.stream == 5 # Specific TCP stream number
```

UDP Filters

```
udp.port == 53 # DNS

udp.port == 67 or udp.port == 68 # DHCP

udp.srcport == 5004 # RTP source

udp.length > 100 # Large UDP datagrams
```

HTTP Filters

```
http # All HTTP traffic

http.request # HTTP requests only

http.response # HTTP responses only

http.request.method == "GET" # GET requests

http.request.method == "POST" # POST requests

http.request.uri contains "/login" # Login page requests

http.response.code == 200 # 200 OK responses

http.response.code == 404 # 404 Not Found

http.response.code >= 400 # All error responses

http.host == "example.com" # Specific host

http.cookie # Packets with cookies

http.user_agent contains "curl" # Requests from curl

http.content_type contains "json" # JSON responses

http contains "password" # Raw text search (DANGEROUS!)
```

DNS Filters

```
dns # All DNS traffic

dns.qry.name == "example.com" # Specific query

dns.qry.name contains ".ru" # Russian TLD queries

dns.qry.type == 1 # A record queries

dns.qry.type == 28 # AAAA (IPv6) queries

dns.qry.type == 15 # MX record queries

dns.qry.type == 16 # TXT record queries

dns.resp.ttl < 60 # Suspiciously low TTL

dns.flags.rcode == 3 # NXDOMAIN responses

dns.count.answers > 5 # Many answers (DNS amplification?)
```

TLS / HTTPS Filters

```
tls # All TLS traffic

tls.handshake # TLS handshake packets

tls.handshake.type == 1 # ClientHello

tls.handshake.type == 2 # ServerHello

tls.record.version == 0x0303 # TLS 1.2

tls.record.version == 0x0304 # TLS 1.3

ssl.handshake.ciphersuite == 0x002f # Specific cipher suite

tls.handshake.extensions_server_name # SNI field present
```

ARP Filters

```
arp # All ARP traffic

arp.opcode == 1 # ARP Request

arp.opcode == 2 # ARP Reply

arp.src.hw_mac == aa:bb:cc:dd:ee:ff # From specific MAC

arp.duplicate-address-detected # Duplicate IP detected

arp.dst.hw_mac == 00:00:00:00:00:00 # Broadcast ARP request
```

ICMP Filters

```
icmp # All ICMP

icmp.type == 8 # Echo Request (ping)

icmp.type == 0 # Echo Reply

icmp.type == 3 # Destination Unreachable

icmp.type == 11 # Time Exceeded (TTL expired – traceroute)
```

Malware & Security Filters

```
# Long DNS queries (DNS tunnelling)

dns.qry.name.len > 50

# Beacons: regular intervals to one host (look in statistics)
```

```
ip.dst == <suspected C2 IP>

# Non-standard HTTP ports

tcp.port != 80 and tcp.port != 443 and http

# SYN flood detection

tcp.flags.syn == 1 and tcp.flags.ack == 0

# Port scan: many RST responses

tcp.flags.rst == 1

# Data exfiltration: large outbound packets

ip.src == 192.168.1.0/24 and ip.len > 1200

# NBNS / LLMNR poisoning

nbns or llmnr

# Pass-the-hash / SMB auth

smb or smb2
```

6.3 Building Filters from Right-Click

- Click a packet → expand a protocol field in the Details panel.
- Right-click the field → **Apply as Filter** → **Selected**.
- Wireshark automatically builds the correct filter expression and applies it.
- Use **Prepare as Filter** to build without immediately applying (allows editing first).
- Use **And Selected** / **Or Selected** / **And Not Selected** to combine with existing filter.

6.4 The Filter Expression Dialog

Click the bookmark icon in the filter bar → **Manage Display Filters** to save, name, and organise your favourite filters. The **Filter Expression** dialog (accessible via the + button in the filter bar) provides a searchable tree of all thousands of available filter fields with descriptions.

■ PRO TIP

Press Ctrl+/ or click in the filter bar and press Tab for autocomplete suggestions.

Wireshark shows all matching field names as you type — extremely useful for discovery.

Type 'ip.' and see every IP-layer field available.

Protocol Analysis — The TCP/IP Stack

Understanding how Wireshark decodes each protocol layer is essential. We'll walk through the full TCP/IP stack layer by layer.

7.1 Layer 2: Ethernet Frames

The **Ethernet II** frame is the outermost layer in most LAN captures:

- **Destination MAC** (6 bytes) — recipient's hardware address. FF:FF:FF:FF:FF:FF = broadcast.
- **Source MAC** (6 bytes) — sender's hardware address. First 3 bytes = OUI (manufacturer).
- **EtherType** (2 bytes) — 0x0800 = IPv4, 0x0806 = ARP, 0x86DD = IPv6, 0x8100 = VLAN.
- **Payload** — the encapsulated higher-layer protocol.
- **FCS** (4 bytes) — Frame Check Sequence (CRC). Usually stripped by the NIC driver.

Wireshark resolves OUI prefixes to manufacturer names automatically (View → Name Resolution → Resolve MAC Addresses).

7.2 Layer 3: IPv4 Header Analysis

Every IPv4 packet has a 20-byte (minimum) header. Key fields visible in Wireshark:

- **Version** — 4 for IPv4.
- **IHL** — Internet Header Length in 32-bit words (5 = 20 bytes, no options).
- **DSCP / ECN** — Differentiated Services / Explicit Congestion Notification (QoS).
- **Total Length** — total packet size including header and payload.
- **Identification** — used for fragment reassembly.
- **Flags** — bit 0: reserved; bit 1: DF (Don't Fragment); bit 2: MF (More Fragments).
- **Fragment Offset** — position of fragment in original datagram.
- **TTL** — Time To Live. Decrement by each router. OS defaults: Linux=64, Windows=128, Cisco=255.
- **Protocol** — 6=TCP, 17=UDP, 1=ICMP, 89=OSPF, 47=GRE.
- **Header Checksum** — validated by Wireshark (green = valid, red = bad).
- **Source / Destination IP** — 32-bit addresses.
- **Options** — rarely used in modern traffic; presence may indicate scanning.

7.3 TCP Three-Way Handshake

TCP establishes a connection with a three-way handshake. To see it in Wireshark:

```
# Filter for handshake of a specific connection

tcp.flags.syn == 1 or tcp.flags.fin == 1 or tcp.flags.rst == 1

# Or filter by stream number (right-click → Follow → TCP Stream to find stream ID)

tcp.stream == 0
```

The sequence:

- **SYN** — Client → Server: SYN flag set, sequence number X (ISN), window size advertised.
- **SYN-ACK** — Server → Client: SYN+ACK flags, sequence Y, acknowledgement X+1.
- **ACK** — Client → Server: ACK flag, acknowledgement Y+1. Connection established.

TCP termination (graceful close — FIN/ACK exchange) and abrupt reset (RST) are also visible. Wireshark labels RSTs in red — always investigate them.

7.4 TCP Flags Reference

Flag	Bit	Meaning
SYN	0x0 02	Synchronise — initiate connection
ACK	0x0 10	Acknowledge received data
FIN	0x0 01	Finish — no more data to send
RST	0x0 04	Reset — abort connection immediately
PSH	0x0 08	Push — deliver data immediately (no buffering)
URG	0x0 20	Urgent pointer field is significant
ECE	0x0 40	ECN-Echo — congestion notification

CWR	0x0 80	Congestion Window Reduced
NS	0x1 00	ECN Nonce Sum (experimental)

7.5 TCP Expert Information

Wireshark's TCP dissector automatically analyses sequences and raises expert items:

- **TCP Retransmission** — segment resent (packet loss). Filter: **tcp.analysis.retransmission**
- **Fast Retransmission** — resent after 3 duplicate ACKs. Filter: **tcp.analysis.fast_retransmission**
- **Duplicate ACK** — receiver ACKing already-ACKed data (signals gap). Filter: **tcp.analysis.duplicate_ack**
- **Out-of-Order** — segment arrived out of sequence. Filter: **tcp.analysis.out_of_order**
- **Zero Window** — receiver buffer full, pausing sender. Filter: **tcp.analysis.zero_window**
- **Window Full** — sender has filled receiver's window. Filter: **tcp.analysis.window_full**
- **Keep-Alive** — periodic zero/one-byte probes to detect dead connections.

Following Streams

One of Wireshark's most powerful features is stream reconstruction. It reassembles all packets of a conversation into a human-readable dialogue.

8.1 Follow TCP Stream

- Right-click any packet in a TCP conversation.
- Select **Follow** → **TCP Stream** (or **Analyse** → **Follow** → **TCP Stream**).
- A new window shows the complete conversation: client data in **red**, server in **blue**.
- Use the **Stream** spinner at the bottom to jump between different TCP streams.
- Change the **Show data as** dropdown: ASCII, EBCDIC, Hex Dump, C Arrays, Raw, YAML.
- Click **Save As** to export the raw stream bytes.
- Notice the display filter bar automatically updates to: **tcp.stream eq N**

■ PRO TIP

Use 'Follow TCP Stream' to instantly read plaintext HTTP requests and responses, extract credentials from unencrypted protocols (Telnet, FTP, POP3, SMTP), and reconstruct downloaded files or uploaded data.

8.2 Follow UDP Stream

Same as TCP stream but for UDP conversations (DNS, VoIP RTP, DHCP, etc.). Right-click → **Follow** → **UDP Stream**. Useful for examining DNS query/response pairs and detecting DNS tunnelling payloads.

8.3 Follow TLS Stream

For encrypted TLS traffic, Follow TLS Stream shows the decrypted application data — **only if** you have provided the session keys or server private key.

- **Method 1** — **Pre-Master Secret Log**: Set environment variable **SSLKEYLOGFILE=/path/to/keys.log** before launching Chrome/Firefox. The browser writes TLS session keys to this file.

- In Wireshark: Edit → Preferences → Protocols → TLS → **(Pre)-Master-Secret log filename** → point to keys.log.
- **Method 2 — Server RSA Private Key:** Works only with RSA key exchange (no PFS). Go to Edit → Preferences → Protocols → TLS → RSA keys list → Add the server's private .pem key.
- After configuration, HTTP/2 and HTTP/1.1 traffic over TLS becomes readable.

```
# On Linux – capture browser TLS keys

export SSLKEYLOGFILE=~/tls_keys.log

google-chrome &

# Start Wireshark capture, browse, stop capture

# Configure Wireshark with the key log file

# Now you can Follow TLS Stream to read HTTPS in plaintext
```

8.4 Follow HTTP Stream

Follow → **HTTP Stream** reconstructs and decompresses HTTP conversations, including chunked encoding and gzip-compressed responses. This is much easier to read than raw TCP stream when dealing with HTTP/1.1 pipelining.

8.5 Extracting Files from Streams

Wireshark can extract transferred files directly: **File** → **Export Objects** → **HTTP** (also SMB, TFTP, DICOM, IMF). A dialog lists all objects transferred — click any to save or preview. This is invaluable for malware analysis (extracting EXE/DLL dropped via HTTP) and forensics (recovering documents exfiltrated over SMB).

HTTP & HTTPS Analysis

9.1 Analysing Plain HTTP

HTTP/1.1 is entirely plaintext and trivially readable. Apply the filter **http** and select any HTTP request packet.

In the Packet Details panel, expand **Hypertext Transfer Protocol** to see:

- **Request Method & URI** — GET /index.html HTTP/1.1
- **Host header** — the virtual host requested.
- **User-Agent** — browser / client software string (fingerprinting).
- **Cookie** — session tokens, authentication cookies (CRITICAL for security).
- **Authorization** — Basic auth credentials (base64 encoded — trivially decoded).
- **Content-Type / Content-Length** — body format and size.
- **Referer** — previous page (navigation path analysis).

9.2 Decoding Basic Auth

HTTP Basic Authentication sends credentials as Base64 in the Authorization header. Wireshark decodes it automatically. To find it:

```
# Filter for Basic Auth

http.authorization

# Wireshark shows: Authorization: Basic dXNlcjpwYXNzd29yZA==

# Decoded: user:password

# Verify in terminal

echo 'dXNlcjpwYXNzd29yZA==' | base64 -d
```

9.3 HTTP/2 Analysis

HTTP/2 is binary and multiplexed. Wireshark decodes it fully, showing stream IDs, HEADERS frames (compressed with HPACK), DATA frames, PUSH_PROMISE, etc. Filter: **http2**. Wireshark automatically rebuilds HPACK-compressed headers.

9.4 Detecting Web Attacks in HTTP Traffic

```
# SQL Injection in GET parameters

http.request.uri contains "'" or http.request.uri contains "UNION"

# XSS attempts

http.request.uri contains "<script" or http.request.uri contains "alert("

# Directory traversal

http.request.uri contains "../"

# Scanning tools (Nikto, sqlmap, Burp)

http.user_agent contains "Nikto" or http.user_agent contains "sqlmap"

# Unusual HTTP methods

http.request.method == "PUT" or http.request.method == "DELETE"

# Large POST (data exfiltration or file upload)

http.request.method == "POST" and tcp.len > 10000
```

9.5 HTTPS / TLS Handshake Deep Dive

Even without decryption keys, TLS metadata reveals a lot:

- **SNI (Server Name Indication)** — Filter: `tls.handshake.extensions_server_name` shows the hostname the client is connecting to, even over TLS.
- **Certificate Information** — In ServerHello/Certificate packets, expand TLS → Handshake Protocol → Certificate to see CN, SAN, issuer, validity dates, and fingerprint.
- **Cipher Suite Negotiation** — weak ciphers (RC4, DES, EXPORT) are a red flag.
- **TLS Version** — TLS 1.0/1.1 are deprecated. SSLv3 = critical vulnerability.
- **Self-Signed Certs** — malware often uses self-signed or mismatched certificates for C2.
- **JA3 Fingerprint** — TShark can compute JA3 hashes from ClientHello to fingerprint client apps and malware families.

■ PRO TIP

The SNI field (tls.handshake.extensions_server_name) shows the destination hostname for HTTPS traffic even when you cannot decrypt the content. This alone can identify C2 domains, phishing infrastructure, and data exfiltration endpoints.

DNS Analysis

DNS (Domain Name System, UDP/TCP port 53) is fundamental to almost all internet communication and is a favourite vector for attackers — from C2 beaconing to full DNS tunnelling.

10.1 DNS Packet Structure

- **Transaction ID** — 16-bit ID matching query to response.
- **Flags** — QR (query/response), Opcode, AA, TC, RD, RA, RCODE.
- **Questions** — the domain name queried and record type.
- **Answers** — resolved records (A, AAAA, CNAME, MX, TXT, etc.).
- **Authority** — NS records for the authoritative nameserver.
- **Additional** — extra resource records (glue records, OPT for EDNS).

10.2 Common DNS Record Types

Type	ID	Purpose
A	1	IPv4 address mapping
AAAA	2 8	IPv6 address mapping
CNAME	5	Canonical name alias
MX	1 5	Mail exchange server
NS	2	Name server record
TXT	1 6	Arbitrary text (SPF, DKIM, verification)
PTR	1 2	Reverse DNS lookup
SOA	6	Start of Authority
SRV	3 3	Service locator

10.3 Detecting DNS Tunnelling

DNS tunnelling encodes data in DNS queries/responses to exfiltrate data or bypass firewalls:

- Look for **unusually long subdomain labels** (normal <63 chars per label; tunnelling may hit 63).
- Watch for **high query frequency** to a single domain (beaconing pattern).
- Look for **TXT or NULL record type queries** — rarely used legitimately in high volumes.
- Suspicious: random-looking subdomains like **aGVsbG8gd29ybGQ.evil.com** (base64 encoded).
- High NXDOMAIN rate — DGA domains resolve to nothing.

```
# Long DNS query names (> 50 chars)

dns.qry.name.len > 50

# High-frequency queries (use Statistics → DNS to see counts)

# TXT record queries (often used in tunnelling)

dns.qry.type == 16

# NXDOMAIN flood (DGA malware)

dns.flags.rcode == 3

# Queries matching base64 pattern (Wireshark regex)

dns.qry.name matches "[A-Za-z0-9+/]{20,}"
```

10.4 DNS Statistics

Go to **Statistics** → **DNS** for a dashboard showing: request types, response codes, query name lengths, service/query counts, and timing. A spike in NXDOMAIN responses is a classic indicator of Domain Generation Algorithm (DGA) malware.

ARP, ICMP & Network Layer Analysis

11.1 ARP Analysis

ARP (Address Resolution Protocol) maps IP addresses to MAC addresses on a LAN. It has no authentication and is trivially spoofable — making it a favourite attack vector.

- **Gratuitous ARP** — an ARP Reply sent without a request. Used for IP announcement and failover, but also used in ARP poisoning attacks.
- **ARP Poisoning / MITM** — attacker floods the LAN with fake ARP Replies, associating their MAC with the victim's or gateway's IP. All traffic is redirected through the attacker.

```
# View all ARP traffic

arp

# Detect gratuitous ARP (same sender IP = target IP)

arp.opcode == 2 and arp.src.proto_ipv4 == arp.dst.proto_ipv4

# Detect ARP storm (too many ARPs)

arp

# Then Statistics → Conversations → look for huge ARP counts

# ARP poisoning: same IP answered by multiple MACs

# Look for: same arp.src.proto_ipv4 with different arp.src.hw_mac

arp.duplicate-address-detected
```

■ WARNING

If you see the same IP address appearing with two different MAC addresses in the ARP table — that is the hallmark of ARP poisoning / MITM attack. Cross-reference with Statistics → Endpoints → Ethernet to identify the rogue MAC.

11.2 ICMP Analysis

ICMP (Internet Control Message Protocol) provides network diagnostics. Used by ping, traceroute, and many scanning tools.

- **Type 8 / Code 0** — Echo Request (ping sent).
- **Type 0 / Code 0** — Echo Reply (ping response).
- **Type 3** — Destination Unreachable. Code 0=Net, 1=Host, 2=Protocol, 3=Port, 4=Fragmentation needed.
- **Type 11** — Time Exceeded. Code 0=TTL expired (traceroute hop), Code 1=Fragment reassembly timeout.
- **Type 5** — Redirect. A router telling a host to use a different gateway (can be weaponised).
- **Type 13/14** — Timestamp Request/Reply.
- **ICMP Tunnelling** — data encoded in the ICMP payload field (e.g., the iodine tool).

```
# All ICMP

icmp

# Ping flood (DoS)

icmp.type == 8

# Check packet rate in Statistics → I/O Graphs

# Traceroute (TTL-exceeded ICMP)

icmp.type == 11

# Detect ICMP tunnelling (large payload)

icmp and data.len > 64

# ICMP redirect (suspicious if unexpected)

icmp.type == 5
```

11.3 IPv6 Analysis

IPv6 introduces new protocols: ICMPv6, NDP (Neighbour Discovery), MLD. Key filters:

```
ipv6 # All IPv6

icmpv6 # ICMPv6 (replaces ARP with NDP)
```

```
icmpv6.type == 135 # Neighbour Solicitation (like ARP Request)

icmpv6.type == 136 # Neighbour Advertisement (like ARP Reply)

icmpv6.type == 133 # Router Solicitation

icmpv6.type == 134 # Router Advertisement

# IPv6 link-local traffic

ipv6.src starts_with "fe80"
```

Wireless (Wi-Fi) Packet Analysis

Capturing 802.11 traffic requires a wireless adapter in **monitor mode** and a Linux system with the right drivers (Alfa cards are popular for this).

12.1 802.11 Frame Types

- **Management Frames** — Beacon, Probe Request/Response, Authentication, Association, Deauthentication, Disassociation.
- **Control Frames** — RTS, CTS, ACK, Block ACK.
- **Data Frames** — actual data payload (encrypted in WPA2/WPA3).

12.2 Key Wi-Fi Filters

```
wlan # All 802.11 frames

wlan.fc.type == 0 # Management frames

wlan.fc.type == 1 # Control frames

wlan.fc.type == 2 # Data frames

wlan.fc.type_subtype == 0x08 # Beacon frames

wlan.fc.type_subtype == 0x04 # Probe Request

wlan.fc.type_subtype == 0x05 # Probe Response

wlan.fc.type_subtype == 0x0b # Authentication

wlan.fc.type_subtype == 0x0c # Deauthentication

wlan.bssid == aa:bb:cc:dd:ee:ff # Specific AP

wlan.ssid == "MyNetwork" # Specific SSID

eapol # WPA handshake frames
```

12.3 Capturing the WPA/WPA2 Handshake

The 4-way EAPOL handshake is captured when a client authenticates to an AP. This can be used (offline) to test passphrase strength:

```
# Step 1: Put adapter in monitor mode on target channel

sudo airmon-ng start wlan0

sudo airodump-ng -c 6 --bssid AA:BB:CC:DD:EE:FF wlan0mon

# Step 2: Open the saved .pcap in Wireshark

# Filter for EAPOL frames

eapol

# You should see 4 EAPOL packets (Message 1/4, 2/4, 3/4, 4/4)

# This is the WPA 4-way handshake

# Step 3: Convert to hashcat format for password testing

hcxpcapngtool -o hash.hc22000 capture.pcapng

hashcat -m 22000 hash.hc22000 wordlist.txt
```

■ WARNING

Capturing Wi-Fi handshakes and testing passphrases is illegal on networks you do not own or have explicit permission to test. Only do this on your own lab network.

12.4 Detecting Deauth Attacks

Deauthentication attacks flood clients with spoofed deauth frames, forcing them to reconnect (enabling handshake capture):

```
# Filter for deauthentication frames

wlan.fc.type_subtype == 0x0c

# If you see a high rate of deauth frames – attack in progress

# Identify attacker: check wlan.ta (Transmitter Address)

# Legitimate deauths: occasional, from AP MAC

# Attack: rapid burst, often from spoofed or random MAC
```

Statistics & I/O Graphs

13.1 Protocol Hierarchy (Statistics → Protocol Hierarchy)

Shows a tree breakdown of all protocols in the capture with packet count, byte count, and percentage. Instantly reveals: what fraction of traffic is TCP vs UDP, HTTP vs TLS, DNS, etc. Useful for baselining and anomaly detection.

13.2 Conversations (Statistics → Conversations)

Lists all unique communication pairs at multiple layers (Ethernet, IPv4, IPv6, TCP, UDP). Columns: address A, port A, address B, port B, packets, bytes, duration. Sort by bytes to find the heaviest talkers. Sort by packets to find port scanners.

13.3 Endpoints (Statistics → Endpoints)

Per-host statistics. Clicking **Map** displays a world map with endpoint geolocations (requires GeoIP database — see Edit → Preferences → Name Resolution → GeoIP). Instantly spot connections to foreign IPs that should not be there.

13.4 I/O Graphs (Statistics → I/O Graphs)

Time-series line charts of packet rate or byte rate. Key features:

- Add multiple curves with different display filters to compare traffic types.
- Change Y-axis to **Bits/s** for bandwidth analysis.
- Change interval (X-axis resolution) from default 1s to 0.1s for fine-grained spikes.
- Use **SUM(field)** style to graph specific values (e.g., TCP window size over time).
- Example — plot SYN flood: add filter **tcp.flags.syn==1 and tcp.flags.ack==0**, watch for a sudden spike that immediately precedes service disruption.

13.5 TCP Stream Graphs

With a TCP packet selected, go to **Statistics → TCP Stream Graphs**:

- **Time/Sequence (Stevens)** — classic graph showing sequence number vs time. Plateaus = stalls.

- **Time/Sequence (tcptrace)** — shows both directions with ACKs and segments.
- **Throughput** — calculated throughput over time.
- **Round-Trip Time** — RTT per packet. Spikes = congestion.
- **Window Scaling** — receiver window size over time.

13.6 Flow Graph (Statistics → Flow Graph)

Generates a ladder diagram showing the sequence of packets between hosts. Perfect for visualising handshakes, protocol exchanges, and identifying exactly where in a conversation an error occurs.

■ PRO TIP

For incident response, always start with: Protocol Hierarchy → Conversations → Endpoints.

This gives you a 30-second overview of the entire capture before diving into individual packets.

Wireshark for Security — Attack Detection

This chapter is the heart of cybersecurity analysis with Wireshark. We'll cover detecting every major network attack class.

14.1 Port Scanning Detection

Port scans probe many ports quickly. Different scan types leave distinct packet signatures:

- **TCP SYN Scan (nmap -sS)** — attacker sends SYN, never completes handshake. Look for: many SYN packets from one IP to sequential ports, many RST-ACK responses.
- **TCP Connect Scan (nmap -sT)** — full handshake then RST. Look for many completed handshakes in rapid succession.
- **UDP Scan (nmap -sU)** — sends UDP to many ports, gets ICMP Port Unreachable for closed ports.
- **NULL / FIN / Xmas Scan** — unusual flag combinations to bypass firewalls.
- **Idle / Zombie Scan** — uses a third host's IP ID field to infer port state.

```
# Detect SYN scan (many SYNs, no SYN-ACK response)

tcp.flags.syn == 1 and tcp.flags.ack == 0

# Detect many RST responses (closed ports from scan)

tcp.flags.rst == 1

# NULL scan (no flags set)

tcp.flags == 0x000

# Xmas scan (FIN + PSH + URG)

tcp.flags == 0x029

# FIN scan

tcp.flags == 0x001
```

```
# View Conversations to see one IP hitting many destinations/ports

# Statistics → Conversations → TCP → sort by 'Packets'
```

14.2 ARP Poisoning / MITM Detection

```
# Step 1: Filter ARP only

arp

# Step 2: Look for duplicate IP announcement (gratuitous ARP)

arp.opcode == 2 and arp.src.proto_ipv4 == arp.dst.proto_ipv4

# Step 3: Wireshark's built-in detector

arp.duplicate-address-detected

# Step 4: Check Endpoints → Ethernet for MAC addresses

# If two MACs claim the same IP → MITM attack

# Step 5: Look for unusual ARP storm

# Statistics → I/O Graphs → filter: arp → if >10 ARP/sec → suspicious
```

14.3 SYN Flood (DoS) Detection

```
# Many SYN packets, no corresponding ACK completion

tcp.flags.syn == 1 and tcp.flags.ack == 0

# In I/O Graph: plot SYN rate vs connection establishment rate

# SYN rate >> ACK rate = SYN flood

# Check source IPs: random / spoofed = DDoS

# Statistics → Endpoints → IPv4 → sort by packets

# Look for server responses: SYN-ACK rate

tcp.flags.syn == 1 and tcp.flags.ack == 1
```

14.4 DNS Amplification Attack Detection

```
# Large DNS responses (amplification uses ANY or large TXT records)

dns and dns.flags.response == 1 and udp.length > 512

# ANY record queries (common amplification trigger)

dns.qry.type == 255

# Many DNS responses to same destination (victim)

ip.dst == <victim_IP> and dns

# Spoofed source (hard to detect, but response > request ratio is indicator)
```

14.5 LLMNR / NBNS Poisoning (Responder Attacks)

```
# LLMNR – Link-Local Multicast Name Resolution

llmnr

# NBNS – NetBIOS Name Service

nbns

# Responder responds to these with its own IP to capture NTLMv2 hashes

# Look for: LLMNR/NBNS query followed by response from unexpected IP

# Detect Responder's fake responses:

# Source IP responding to LLMNR should be the queried host – if not, suspicious

llmnr and udp.srcport == 5355
```

14.6 SMB Attacks (Pass-the-Hash, EternalBlue)

```
# SMB traffic

smb or smb2

# NTLM authentication over SMB (NTLMv2 hashes)
```

```
ntlmssp

# EternalBlue / MS17-010 indicator: unusual SMB Trans2 requests

smb.cmd == 0x25 # Trans2 command

# SMB brute force: many NTLM authentication failures

# Look for STATUS_LOGON_FAILURE repeated from one source

smb2.status == 0xC000006D # STATUS_LOGON_FAILURE

# SMB file reads (lateral movement / data staging)

smb2.cmd == 5 or smb2.cmd == 8 # Create / Read
```

14.7 Data Exfiltration Detection

```
# Large outbound TCP transfers from internal to external

ip.src == 192.168.0.0/16 and ip.dst != 192.168.0.0/16 and tcp.len > 1000

# HTTP POST with large body (data upload/exfiltration)

http.request.method == "POST" and tcp.len > 5000

# DNS exfiltration (long queries)

dns.qry.name.len > 50

# ICMP with data payload (ICMP tunnel)

icmp.type == 8 and data.len > 64

# FTP data transfers

ftp-data

# Unexpected ports with high data volume

# Statistics → Conversations → TCP → sort by bytes → check unfamiliar ports
```

14.8 C2 Beacon Detection

C2 (Command and Control) beaconing is regular, periodic communication from malware to its operator. Key indicators:

- **Regular intervals** — exact or jittered interval (e.g., every 60 seconds $\pm 5s$).
- **Small, fixed-size packets** — heartbeat check-ins are typically small.
- **Unusual hours** — traffic at 3 AM from a workstation.
- **Odd destination** — IP in unusual geographic location, no PTR record.
- **Non-standard ports** — HTTPS on port 443 but cert is self-signed.
- **Low TTL DNS records** — fast-flux infrastructure.

```
# Filter suspected C2 IP

ip.addr == <suspected_C2_IP>

# Use I/O Graphs to visualise beacon interval

# Statistics → I/O Graphs → add filter for C2 IP

# Regular spikes = beacon

# Check TLS certificate details

tls.handshake.type == 11 # Certificate message

# Expand: check CN, issuer, not before/after dates

# Self-signed or expired cert from 'legitimate' site = red flag
```

Malware Traffic Analysis

Malware analysis with Wireshark involves examining .pcap files from sandboxes (ANY.RUN, VirusTotal, Cuckoo) or live captures in isolated lab environments.

15.1 Malware Analysis Workflow

- **Step 1** — Open the malware PCAP from a sandbox report.
- **Step 2** — Check Protocol Hierarchy for unusual protocols.
- **Step 3** — Check Conversations for destination IPs and sort by bytes.
- **Step 4** — Run IOC filters (suspicious domains, IPs, User-Agents).
- **Step 5** — Follow TCP/HTTP streams to see C2 commands and responses.
- **Step 6** — Extract Files: File → Export Objects → HTTP (look for EXE, DLL, PS1).
- **Step 7** — Check DNS for DGA patterns or C2 domain lookups.
- **Step 8** — Examine TLS certificates for suspicious issuers.
- **Step 9** — Document all IOCs: IPs, domains, file hashes, URIs, User-Agents.

15.2 Detecting Common Malware Families

Emotet / Qakbot (Banking Trojans)

```
# Emotet uses HTTP for C2, often on random high ports

http and tcp.dstport != 80 and tcp.dstport != 443

# Look for POST to random URI patterns

http.request.method == "POST" and http.request.uri matches "[a-z]{4,10}"/

# Emotet drops additional payloads – extract via File → Export Objects → HTTP
```

Cobalt Strike (Red Team / APT tool)

```
# Default Cobalt Strike beacon: HTTPS to random-looking domain

# Check TLS SNI
```

```
tls.handshake.extensions_server_name

# Beacon checks in every 60s by default (jitter can be set)

# Size often 212 bytes for check-in

# HTTP malleable profile – can mimic legitimate traffic

# Look for: consistent User-Agent, fixed URI patterns, regular timing

# DNS C2 mode

dns.qry.name contains "stager" # Look for stage download
```

Ransomware

```
# Pre-encryption: reconnaissance (SMB shares enumeration)

smb2.cmd == 5 # Tree Connect

# Key exchange to C2 before encryption

tls or http # C2 checkin / key fetch

# Post-encryption: C2 exfil or ransom note upload

# Large outbound HTTP POST

http.request.method == "POST" and tcp.len > 10000

# Lateral movement before encryption

# Many SMB connections from one host to entire subnet

smb and ip.src == <infected_host>
```

15.3 JA3 / JA3S Fingerprinting

JA3 is an MD5 hash of selected TLS ClientHello fields. Different malware families produce consistent JA3 hashes regardless of IP or domain. JA3S fingerprints server responses. Both can be added to Wireshark with the **ja3.lua** plugin or computed via TShark:

```
# Compute JA3 hashes with TShark
```



```
tshark -r capture.pcap -Y tls.handshake.type==1 \  
-T fields -e ip.src -e tls.handshake.ja3  
  
# Check computed hash against known-bad JA3 database:  
  
# https://sslbl.abuse.ch/ja3-fingerprints/
```

TShark — Command Line Wireshark

TShark is the command-line equivalent of Wireshark. It supports the same capture and display filters, decodes the same protocols, and is essential for scripting, automation, and headless server analysis.

16.1 Basic TShark Commands

```
# List available interfaces

tshark -D

# Capture on interface eth0

tshark -i eth0

# Capture 100 packets then stop

tshark -i eth0 -c 100

# Capture with BPF filter

tshark -i eth0 -f 'port 80'

# Save to file

tshark -i eth0 -w capture.pcap

# Read from file

tshark -r capture.pcap

# Apply display filter to file

tshark -r capture.pcap -Y 'http.request'

# Verbose packet detail

tshark -r capture.pcap -V
```

```
# Summary only (one line per packet)

tshark -r capture.pcap -q -z io,stat,1
```

16.2 Field Extraction with -T fields

```
# Extract source IP, destination IP, and protocol

tshark -r capture.pcap -T fields -e ip.src -e ip.dst -e ip.proto


# Extract HTTP host and URI

tshark -r capture.pcap -Y http.request -T fields \

-e http.host -e http.request.uri -e http.request.method


# Extract DNS queries

tshark -r capture.pcap -Y dns.qry.name -T fields \

-e frame.time -e ip.src -e dns.qry.name -e dns.qry.type


# Extract all TLS SNI values

tshark -r capture.pcap -Y 'tls.handshake.type==1' -T fields \

-e ip.dst -e tls.handshake.extensions_server_name


# Extract TCP stream contents

tshark -r capture.pcap -q -z follow,tcp,ascii,0


# Output as JSON

tshark -r capture.pcap -T json > output.json


# Output as CSV

tshark -r capture.pcap -T fields -e frame.number -e ip.src -e ip.dst \

-E header=y -E separator=, -E quote=d > output.csv
```

16.3 TShark Statistics Commands

```
# I/O statistics – packet count per second

tshark -r capture.pcap -q -z io,stat,1


# Protocol hierarchy

tshark -r capture.pcap -q -z io,phs


# Conversation statistics (TCP)

tshark -r capture.pcap -q -z conv,tcp


# Endpoint statistics

tshark -r capture.pcap -q -z endpoints,ip


# HTTP request / response statistics

tshark -r capture.pcap -q -z http,tree


# DNS statistics

tshark -r capture.pcap -q -z dns,tree


# Export objects

tshark -r capture.pcap --export-objects http://tmp/exported_files
```

16.4 Practical TShark Scripts

```
#!/bin/bash

# Script: Extract all unique destination IPs contacted during capture

tshark -r suspicious.pcap -T fields -e ip.dst \
2>/dev/null | sort -u | grep -v '^$' > contacted_ips.txt

echo "[+] Unique destination IPs:"

wc -l contacted_ips.txt


# Script: Extract all unique DNS queries from a capture
```

```
tshark -r suspicious.pcap -Y 'dns.flags.response==0' \  
-T fields -e dns.qry.name 2>/dev/null | sort -u > dns_queries.txt  
  
echo "[+] Unique DNS queries:"  
  
wc -l dns_queries.txt
```

16.5 Combining TShark with Other Tools

```
# Pipe TShark output to grep for quick IOC hunting  
  
tshark -r capture.pcap -T fields -e http.host 2>/dev/null | \  
grep -i 'evil\|malware\|suspicious'  
  
# Pipe to Python for advanced analysis  
  
tshark -r capture.pcap -T json 2>/dev/null | python3 analyze.py  
  
# Use with jq for JSON processing  
  
tshark -r capture.pcap -T json 2>/dev/null | \  
jq '._source.layers.http.host // empty'  
  
# Count packets per destination port (quick port survey)  
  
tshark -r capture.pcap -T fields -e tcp.dstport 2>/dev/null | \  
sort | uniq -c | sort -rn | head -20
```

Exporting, Reporting & Evidence Handling

17.1 Saving Captures

- **.pcapng** (default, recommended) — pcap Next Generation. Supports metadata, interface info, multiple sections, comments.
- **.pcap** — legacy libpcap format. Maximum compatibility with third-party tools.
- **.cap** — same as .pcap, common on Cisco and older tools.
- File → Save As → select format. Use **Export Specified Packets** to save only filtered packets.

17.2 Export Formats

```
# From Wireshark GUI: File → Export Packet Dissections

# Formats:

# Plain Text (.txt) — one line per packet

# PostScript (.ps) — printable format

# CSV (.csv) — spreadsheet analysis

# JSON (.json) — programmatic processing

# XML (PSML) (.xml) — packet summary

# XML (PDML) (.xml) — full packet details (verbose)


# From TShark:

tshark -r capture.pcap -T json > report.json

tshark -r capture.pcap -T pdml > report.xml

tshark -r capture.pcap -T fields -e frame.number -e ip.src \
-e ip.dst -e tcp.dstport -E header=y -E separator=, > report.csv
```

17.3 Exporting Objects (File Carving)

- Go to **File** → **Export Objects**.
- Choose protocol: **HTTP**, **SMB**, **TFTP**, **DICOM**, **IMF**.
- A dialog lists all transferred objects with filename, content type, and size.
- Select individual files or **Save All** to a directory.
- Check extracted files: **file <filename>** to verify type, then hash with **md5sum** / **sha256sum**.
- Submit hashes to VirusTotal for malware identification.

17.4 Evidence Handling for Forensics

When using Wireshark captures as digital evidence, follow these practices:

- **Integrity** — generate SHA-256 hash of the raw .pcapng file immediately after capture. Store hash separately.
- **Chain of Custody** — document who captured, on which interface, at what time (use NTP-synchronised clock).
- **Timestamps** — set Wireshark to UTC (View → Time Display Format → UTC Date and Time). Embed in filename: **capture_2024-01-15_14-30-00_UTC.pcapng**.
- **Write Protection** — after capture, set file to read-only (**chmod 444**) or store on write-protected media.
- **Don't modify originals** — always work from copies. Apply display filters (non-destructive) not capture filters to originals.
- **Annotate** — use packet comments (right-click → Packet Comment) to annotate significant packets. Saved in pcapng format.
- **Report generation** — export to PDML XML for comprehensive machine-readable report.

```
# Hash the capture file for evidence integrity

sha256sum evidence_capture.pcapng > evidence_capture.sha256

cat evidence_capture.sha256

# Store both files together, log in chain-of-custody form

# Verify integrity later

sha256sum -c evidence_capture.sha256
```

Advanced Features & Plugins

18.1 Lua Scripting

Wireshark has a built-in Lua interpreter. You can write custom dissectors, post-dissectors, and listeners. Place .lua files in the Wireshark plugins directory (Help → About → Folders → Personal Plugins).

```
-- Example: Simple Lua post-dissector that flags large DNS queries

local dns_qry_name = Field.new('dns.qry.name')

local p = Proto('dns_alert', 'DNS Alert Post-Dissector')

function p.dissector(tvb, pinfo, tree)

    local name = dns_qry_name()

    if name and #tostring(name) > 50 then

        local subtree = tree:add(p, 'DNS Alert: Suspiciously long query!')

        pinfo.cols.info:prepend('[LONG DNS] ')

    end

end

register_postdissector(p)
```

18.2 Decode As

Wireshark uses port numbers to guess the protocol. When a service runs on a non-standard port, use **Analyse → Decode As**:

- Right-click a packet → **Decode As**.
- Select the layer (link, network, transport) and the protocol to use.
- Example: if an HTTP proxy runs on port 8080, tell Wireshark to decode TCP port 8080 as HTTP.
- Changes are remembered per-session and can be saved to a profile.

18.3 Coloring Rules

Go to **View** → **Coloring Rules** to create custom colour rules. Each rule has a name, filter expression, foreground colour, and background colour. Rules are processed top-to-bottom (first match wins). Useful custom rules:

- **Red bg: tcp.analysis.retransmission** — highlights retransmissions.
- **Dark red bg: tcp.flags.rst == 1** — highlights RST resets.
- **Orange bg: dns.flags.rcode == 3** — NXDOMAIN responses.
- **Purple bg: http contains "password" or http contains "passwd"** — credential in plaintext!
- **Bright green bg: ip.addr == <your_target_IP>** — highlight specific host.

18.4 Expert Information (Analyse → Expert Information)

Wireshark automatically categorises notable events:

- **Errors (red)** — malformed packets, checksum failures, reassembly errors.
- **Warnings (yellow)** — retransmissions, duplicate ACKs, out-of-order, zero window.
- **Notes (cyan)** — Keep-alives, window updates, ACKed unseen segments.
- **Chats (blue)** — session setup, teardown, protocol handshakes.

The expert info dialog provides a quick health-check of the capture — always review it first when troubleshooting performance issues.

18.5 GeoIP Database Integration

```
# Download MaxMind GeoLite2 databases (free, requires account):

# GeoLite2-City.mmdb, GeoLite2-Country.mmdb, GeoLite2-ASN.mmdb

# Configure in Wireshark:

# Edit → Preferences → Name Resolution → MaxMind database directories

# Point to directory containing the .mmdb files

# Now Statistics → Endpoints → Map shows geographic locations

# IP details panel shows: Country, City, ASN, Latitude/Longitude

# TShark field access:

tshark -r capture.pcap -T fields -e ip.dst \

-e ip.geoip.country -e ip.geoip.city
```

18.6 Custom Columns

Add protocol-specific columns for efficient analysis without opening every packet. Right-click the column header → **Column Preferences**:

- Add **http.host** column to see web destinations at a glance.
- Add **dns.qry.name** to see DNS queries inline.
- Add **tcp.stream** to quickly identify stream numbers.
- Add **ip.ttl** to watch for OS fingerprinting or TTL manipulation.
- Add **frame.time_delta** to see inter-packet timing (beacon detection).

Real-World Labs & CTF Scenarios

19.1 Lab 1: Analyse an HTTP Login

Objective: Find the username and password submitted via HTTP Basic Auth or form POST.

```
# Step 1: Open the pcap

File → Open → http_login.pcap

# Step 2: Apply filter for POST requests

http.request.method == "POST"

# Step 3: Click the POST packet → expand HTTP → look for:

# html form data: username=admin&password=secret123

# OR Authorization: Basic <base64>

# Step 4: For Basic Auth, copy the base64 string

echo 'YWRtaW46c2VjcmV0MTIz' | base64 -d

# Output: admin:secret123

# Step 5: Follow TCP Stream to see the full session

# Right-click → Follow → TCP Stream
```

19.2 Lab 2: Reconstruct a Downloaded File

```
# Step 1: Open pcap containing file download

File → Open → file_transfer.pcap

# Step 2: Export the file

File → Export Objects → HTTP

# Select the file (e.g., suspicious.exe)
```

```
# Save As → /tmp/suspicious.exe

# Step 3: Verify and hash

file /tmp/suspicious.exe

sha256sum /tmp/suspicious.exe

# Step 4: Submit hash to VirusTotal

curl -s 'https://www.virustotal.com/api/v3/files/<hash>' \

-H 'x-apikey: YOUR_API_KEY'
```

19.3 Lab 3: Identify a Port Scan

```
# Step 1: Open the pcap

# Step 2: Statistics → Conversations → TCP

# Sort by 'Packets' – scanner will show one IP talking to many ports

# Step 3: Apply filter for SYN-only packets from scanner IP

ip.src == <scanner_ip> and tcp.flags.syn == 1 and tcp.flags.ack == 0

# Step 4: Count unique destination ports

tshark -r scan.pcap -Y 'ip.src==<scanner_ip> and tcp.flags==0x002' \

-T fields -e tcp.dstport | sort -un | wc -l

# Step 5: Determine scan type by flag pattern

tcp.flags == 0x000 # NULL scan

tcp.flags == 0x029 # Xmas scan

tcp.flags == 0x001 # FIN scan

tcp.flags == 0x002 # SYN scan
```

19.4 Lab 4: Detect ARP Poisoning

```
# Step 1: Filter ARP
```

```
arp

# Step 2: Look for Wireshark expert info

arp.duplicate-address-detected

# Step 3: Find the duplicate

# In Packet Details, note which IP has two different source MACs

# Step 4: Identify legitimate vs attacker MAC

# Check Statistics → Endpoints → Ethernet

# The rogue MAC will have appeared suddenly with many ARP Replies

# Step 5: Trace the attack timeline

# Note timestamp of first fraudulent ARP Reply

# Compare to when normal traffic from victim redirected to attacker
```

19.5 CTF Tips: Pcap Challenges

- **Always start with Protocol Hierarchy** — identifies unusual protocols immediately.
- **Follow every TCP/HTTP stream** — flags are often hidden in plaintext data.
- **Check strings in payloads** — Ctrl+F → String search across all packets.
- **Export all HTTP objects** — flags may be embedded in downloaded images (steganography).
- **Look for encoded data** — base64 in cookies, User-Agents, URIs.
- **Examine DNS names** — flags sometimes encoded in subdomain labels.
- **Check ICMP payloads** — data field often used to hide messages.
- **Decrypt TLS if SSLKEYLOGFILE is provided** — CTF pcaps often come with key logs.
- **Use strings on the raw pcap: `strings capture.pcap | grep -i flag`**
- **Check file magic bytes** of exported objects — PNG hidden as .txt, etc.

Cheat Sheet & Quick Reference

20.1 Essential Keyboard Shortcuts

Shortcut	Action
Ctrl+E	Start / Stop capture
Ctrl+K	Capture options
Ctrl+F	Find a packet
Ctrl+G	Go to packet number
Ctrl+→	Next packet in conversation
Ctrl+←	Previous packet in conversation
Ctrl+Home	First packet
Ctrl+End	Last packet
Ctrl+M	Mark / Unmark packet
Ctrl+D	Scroll down in packets
Ctrl+U	Scroll up in packets
Ctrl+W	Close capture file
Ctrl+S	Save capture
Ctrl+Z	Undo recent change
Ctrl++	Zoom in (increase font)
Ctrl+-	Zoom out
Ctrl+Shift +P	Preferences dialog
Alt+→	Next in selection history
Alt+←	Previous in selection history
F5	Reload (refresh) capture file

20.2 Top 30 Display Filters for Security Analysts

```
# === SCANNING ===

tcp.flags.syn==1 and tcp.flags.ack==0 # SYN scan

tcp.flags==0x000 # NULL scan

tcp.flags==0x029 # Xmas scan

tcp.flags.rst==1 # RST flood


# === CREDENTIALS ===

http.authbasic # HTTP Basic Auth

ftp.request.command == "PASS" # FTP password

telnet # Telnet (plaintext!)

smtp.auth.password # SMTP password

pop.request.command == "PASS" # POP3 password


# === MALWARE IOCs ===

ip.dst == <C2_IP> # C2 traffic

dns.qry.name.len > 50 # DNS tunnel

dns.flags.rcode == 3 # NXDOMAIN (DGA)

tls.handshake.extensions_server_name # SNI (C2 domain)

http and tcp.dstport != 80 and tcp.dstport != 443 # HTTP on odd port


# === NETWORK ATTACKS ===

arp.duplicate-address-detected # ARP poison

icmp.type==8 and ip.src == <attacker> # ICMP flood

smb2.status == 0xC000006D # SMB brute force

nbns or llmnr # Responder attacks
```

```
# === EXFILTRATION ===

http.request.method=="POST" and tcp.len>5000 # HTTP POST exfil

dns.qry.name contains "." # All DNS

ftp-data # FTP data channel


# === TROUBLESHOOTING ===

tcp.analysis.retransmission # Packet loss

tcp.analysis.zero_window # Flow control stall

tcp.analysis.duplicate_ack # Congestion

icmp.type==3 # Unreachable

dns.flags.rcode > 0 # DNS errors

ip.checksum_bad == 1 # Bad checksum
```

20.3 TShark One-Liners Reference

```
# All unique IPs contacted

tshark -r cap.pcap -T fields -e ip.dst | sort -u


# All DNS queries

tshark -r cap.pcap -Y dns -T fields -e dns.qry.name | sort -u


# All HTTP hosts visited

tshark -r cap.pcap -Y http.request -T fields -e http.host | sort -u


# All TLS SNI destinations

tshark -r cap.pcap -Y 'tls.handshake.type==1' \
-T fields -e tls.handshake.extensions_server_name | sort -u


# Live capture + filter + display fields

tshark -i eth0 -Y 'http.request' -T fields \
-e ip.src -e http.host -e http.request.uri
```



```
# Count packets per protocol

tshark -r cap.pcap -q -z io,phs


# Find largest conversations

tshark -r cap.pcap -q -z conv,tcp | sort -k 6 -rn | head -10


# Export all HTTP objects

tshark -r cap.pcap --export-objects http,/tmp/objects/
```

20.4 Wireshark Resources

- **Official Docs:** <https://www.wireshark.org/docs/>
- **Sample PCAPs:** <https://wiki.wireshark.org/SampleCaptures>
- **Malware Traffic Analysis:** <https://www.malware-traffic-analysis.net>
- **Pcap challenges:** <https://packetlife.net/captures/> and CTFtime.org
- **JA3 Hash DB:** <https://sslbl.abuse.ch/ja3-fingerprints/>
- **MaxMind GeoIP:** <https://dev.maxmind.com/geoip/geolite2-free-geolocation-data>
- **ANY.RUN Sandbox PCAPs:** <https://app.any.run>
- **VirusTotal:** <https://virustotal.com> — check extracted file hashes
- **Wireshark Plugins:** <https://gitlab.com/wireshark/wireshark>

ABOUT THE AUTHOR

Tanish Sareen is a Cybersecurity and Digital Forensics specialist pursuing a Bachelor of Computer Applications (BCA) with a specialisation in Cybersecurity. Tanish has hands-on experience building real-world security tools including **ShieldAI** (a published Mozilla Firefox extension for real-time scam detection), **CyberMortem** (an AI-powered cyber forensic intelligence suite), **ForensicOS** (a browser-based offline digital forensics platform), and the **Tanish Ultimate Browser Toolkit (TUBT)** — a comprehensive Firefox MV3 extension combining 18 security and productivity modules.

Tanish is actively involved in the **Institutional Innovation Cell (IIC)**, a Ministry of Education initiative, and has participated in organising national-level startup and investor pitching events. His technical stack spans Python, JavaScript/TypeScript, React, Node.js, Kali Linux, Wireshark, Nmap, and digital forensics tooling.

This ebook represents Tanish's commitment to making practical cybersecurity knowledge accessible to students, professionals, and enthusiasts alike. Every technique documented here has been validated in real lab environments and CTF competitions.

© 2026 Tanish Sareen. All rights reserved. For educational purposes only. Always obtain proper authorisation before performing any network analysis or security testing.